

ACI System Objects Security Standard Requirements

BASE24®



© 2009 by ACI Worldwide Inc. All rights reserved.

All information contained in this documentation, as well as the software described in it, is confidential and proprietary to ACI Worldwide, Inc., or one of its subsidiaries, is subject to a license agreement, and may be used or copied only in accordance with the terms of such license. Except as permitted by such license, no part of this documentation may be reproduced, stored in a retrieval system, or transmitted in any form or by electronic, mechanical, recording, or any other means, without the prior written permission of ACI Worldwide, Inc., or one of its subsidiaries.

ACI, ACI Worldwide, and the ACI product names used in this documentation are trademarks or registered trademarks of ACI Worldwide, Inc., or one of its subsidiaries.

Other companies' trademarks, service marks, or registered trademarks and service marks are trademarks, service marks, or registered trademarks and service marks of their respective companies.

Contents

1	Background.....	5
2	System Object Security Guide	5
2.1	Threats.....	5
2.2	Vulnerabilities	5
2.3	Actors	6
2.3.1	Operators	7
2.3.2	1 st , 2 nd and 3 rd level support	7
2.3.3	Security administrators	7
2.3.4	Database administrators	8
2.3.5	Application software.....	8
2.3.6	External software	8
2.3.7	Audit software	8
2.3.8	Programmed operators	8
2.3.9	Application installers.....	8
2.3.10	Other Actors	9
2.4	System Level Permissions on System Objects.....	10
2.5	Application Specific User IDs and Groups	10
2.6	Generic Diagram.....	10
2.7	Specific Rules for System Objects	12
2.7.1	Temporary Access	13
2.7.2	Transfer of system objects off-site	13
2.7.3	Network Resources	13
2.7.4	Auditing and Audit Logging.....	13
2.7.5	System Object Owner	13
2.7.6	Verification.....	17
3	Command Quick Reference.....	18
3.1	HP NonStop.....	18
3.1.1	XPNET	18
3.1.2	Guardian.....	22
4	Reference and Bibliography	26

ACI Worldwide, Inc.

1 Background

This document is a BASE24-specific guide to security for that application's system objects. It includes a depiction of the security model of BASE24, enumerating the system objects that BASE24 uses, the actors involved in the application, and mapping of access permission between the actors and the system objects.

The guidelines in this document should be carefully reviewed prior to installation of the BASE24 product and where necessary, roles and permissions may need to be modified according to the security policy of the site where the BASE24 product is installed. It is the customer's responsibility to ensure that any such modifications meet the security needs of their site.

2 System Object Security Guide

2.1 Threats

The two types of threats that we are seeking to diminish or eliminate by using the strategies outlined in this guide are:

Theft

Tampering

The basic premise for assessing these two types of threats as priority, is that we must not (can not) assume that all staff who may come into contact with BASE24, are trustworthy.

Other types of threats, including those of network origin (e.g. denial of service attacks), are much less likely for BASE24, due to its lack of exposure to the Internet.

2.2 Vulnerabilities

This section identifies the generic system objects that belong to, are accessed by or are created by BASE24, either directly or indirectly. These system objects are subject to the threats discussed in 2.1 Threats.

Table 1 Vulnerable System Objects

Application Files	Databases	Primary, alternate key and index files Transaction/audit logs and history Database LOAD/UNLOAD utilities' input/output files
	Runtime Files	BASE24 -specific security files ○ Application user id's and passwords ○ Private keys Adhoc and temporary files ○ Scratch/work files ○ Migration/upgrade files ○ Swap files (virtual memory)
	Input Interchange Files	Input files
	Output Interchange Files	Output files Emailed data files Extracts and reports
	Audit Files	Audit logs Database tables for audit
Configuration Files		
Executable Files		Compiled programs DLLs and LIBs Macros Batch control files
Operations Files	Support Files	Dumps/abend files Trace files Event manager log files
	Operations Files	Start/stop/management scripts and jobs
Infrastructure Objects	TP monitors	Pathway servers, etc.
	Message queues	
	Inter-process communication (IPC)	Utilities for shared memory table and semaphore creation/deletion Coupling facility data tables Shared memory Semaphores Pipes
	External devices	communication lines Hardware Security Modules (HSMs)
Installation files		Macros Packed files (e.g. .zip, .tar, .gz, etc.)

2.3 Actors

This section defines the various categories (groups or roles) of users that may potentially be the source of the threats discussed in 2.1 Threats. Each of the groups/roles below refers to identities that are part of the customer site.

2.3.1 Operators

The operator role is responsible for the routine operation of the system and operations such as system backups and recovery or changing recording media. An operator would normally work with many applications, network and system components, perhaps across several systems and possible even platform types, ensuring all are available when required by monitoring their status in event logs and issuing commands to recover application, network and system components that become unavailable.

Several kinds of operator roles may be distinguished. For example batch operator and report operator. However from security viewpoint such a distinction is not required.

An operator role may be combined with any of the other roles.

2.3.2 1st, 2nd and 3rd level support

Support staff are usually called for problem determination when some portion of applications, network or system become unavailable or are not performing as expected. As the names suggest, 1st level support would normally be the first point of contact by operators or business staff having difficulty with system, network or applications.. They perform routine investigative steps to attempt to recover the failed component. 1st level support works at a shallower level, perhaps across several applications or systems. If 1st level support cannot solve a problem it is referred to 2nd level support, which may be an operations support team for a particular platform or range of software applications. If 2nd level support cannot provide a solution, 3rd level support is contacted. 3rd level support for in-house developed products would normally be a support group within the application software's development team. For third party and vendor software, it would be the team that is responsible for its installation and configuration at the customer site. In the case of ACI products, if 3rd level support cannot solve an issue, they would refer the issue to the ACI HELP24 support service.

2.3.3 Security administrators

Security administrators are responsible for configuring the security of the customers' systems as a whole. It is the security administrator who is most familiar with the security system software (e.g. SafeGuard) used at a customer site and who has the ultimate authority within that software.

In this document we do not differentiate between the security administrator's role and any assistants.

2.3.4 Database administrators

The database administrator role is responsible for

- Installation, configuration and maintenance of the application database

- Establishing and maintaining database user accounts.

The database administrator is not allowed to access application data in the database and will not get rights to use the application software. A way to fully implement the desired access restriction is to require that two database administrators each enter half of their password for SYSADM or equivalent access.

2.3.5 Application software

This refers to the ACI-supplied software products that own and/or access the system objects.

2.3.6 External software

Application or system software other than the ACI-supplied software, that requires access to the same system objects as the ACI software.

2.3.7 Audit software

An auditor should not directly be accessing the system objects referred to in this document. Rather, audit software would automatically be monitoring those objects. There are two types of audit trails that are normally used:

- audit records written by the application itself and

- audit software (3rd party or platform-vendor-supplied) which records data about activities on the entire platform.

In both cases, the resulting audit trails/records/logs would need to be viewed by an auditor using special tools (i.e. additional audit software) to summarize/aggregate/interpret the huge amounts of data involved and highlight departures from accepted norms.

2.3.8 Programmed operators

A *programmed* operator is an operations automation term, describing a software program or tool that performs the function of a human operator. Typically, a programmed operator would monitor activity via the event log and take action should certain events occur, e.g. if an event occurs reporting a communications line outage, the programmed operator would issue appropriate communications subsystem commands to stop and start the line in an attempt to get it working again as soon as possible.

2.3.9 Application installers

Application installers are staff who install, configure and update the application software (see 2.3.5 above). During initial installation they would need full access to all application components and system objects. When application upgrades occur, only a subset of the components and system objects may need to be accessed. Application installers would also be responsible for ad hoc updates to system objects used by the application, even if the application itself does not change (e.g. updates to static tables, parameter files, etc.).

2.3.10 Other Actors

All other roles/groups must be denied access on a permanent basis to system objects. This includes the following actors:

2.3.10.1 Application users

Application users do not access system objects directly. Rather, they access the application's web site, GUI, interface panel or other tools. Restrictions to application access are normally controlled by the application itself, typically by managing its own application-level user ids and passwords which are entered on the application's logon screen or window. Often, application-level user ids would *not* be defined to the system's security software (e.g. SafeGuard) so would provide no access to the underlying system. It is always the application software that accesses system objects according to application users' activities within the application.

The sole exception to this rule would be when an executable component of the application software must be started by an application user when they require it, by running that component from the operating system environment (e.g. TACL). In this case, the application users would have an appropriate system-level user id that must be defined in a group that has execute permission on the appropriate system object(s), plus permissions for the other system objects that the executed program must access.

2.3.10.2 System programmers

System programmers, also known as system managers or system administrators, are responsible for the installation, configuration, maintenance, tuning and problem determination for operating system software and utilities. However, there would normally be no need for them to have access to system objects for application software. In the rare case that permissions are required, they can be given appropriate and audited access on an ad-hoc basis.

2.4 System Level Permissions on System Objects

System objects must be secured using operating-system-level security tools such as SafeGuard on HP NonStop to set file permissions on individual objects, or groups of objects, as requirements dictate. Typical permissions that can be set vary from operating system to operating system, but generally include some combination or subset of **read, write, execute, purge**.

Groups of system objects with similar purpose and sensitivity can be secured as one, for example because they reside within a single branch of the file system's hierarchy, or in flat file systems by using a common file name prefix (at the operating system level).

2.5 Application Specific User IDs and Groups

Operating system security tools are used to create unique "identities", or user id's, for each and every individual who must access the system.

All accesses need to be controlled and traceable to an individual, so special privilege user ids shared by more than one individual must not be used. Levels of access can instead be determined by defining roles based on job function, and allocating individual user ids to one or more of these roles. In this way, each individual gains a level of access dictated by the role(s) that he/she has been assigned to.

In the same way, program access to system objects can be controlled, audited and limited by allocating a unique user id to every application.

Each application must be given a unique database user ID and password.

The application audit subsystem must be given its own unique database user id.

2.6 Generic Diagram

The following diagram graphically illustrates the types of permissions on system objects that are appropriate for the various categories of users. Below it, a table maps the generic system objects shown in the diagram, to actual BASE24 system objects' locations

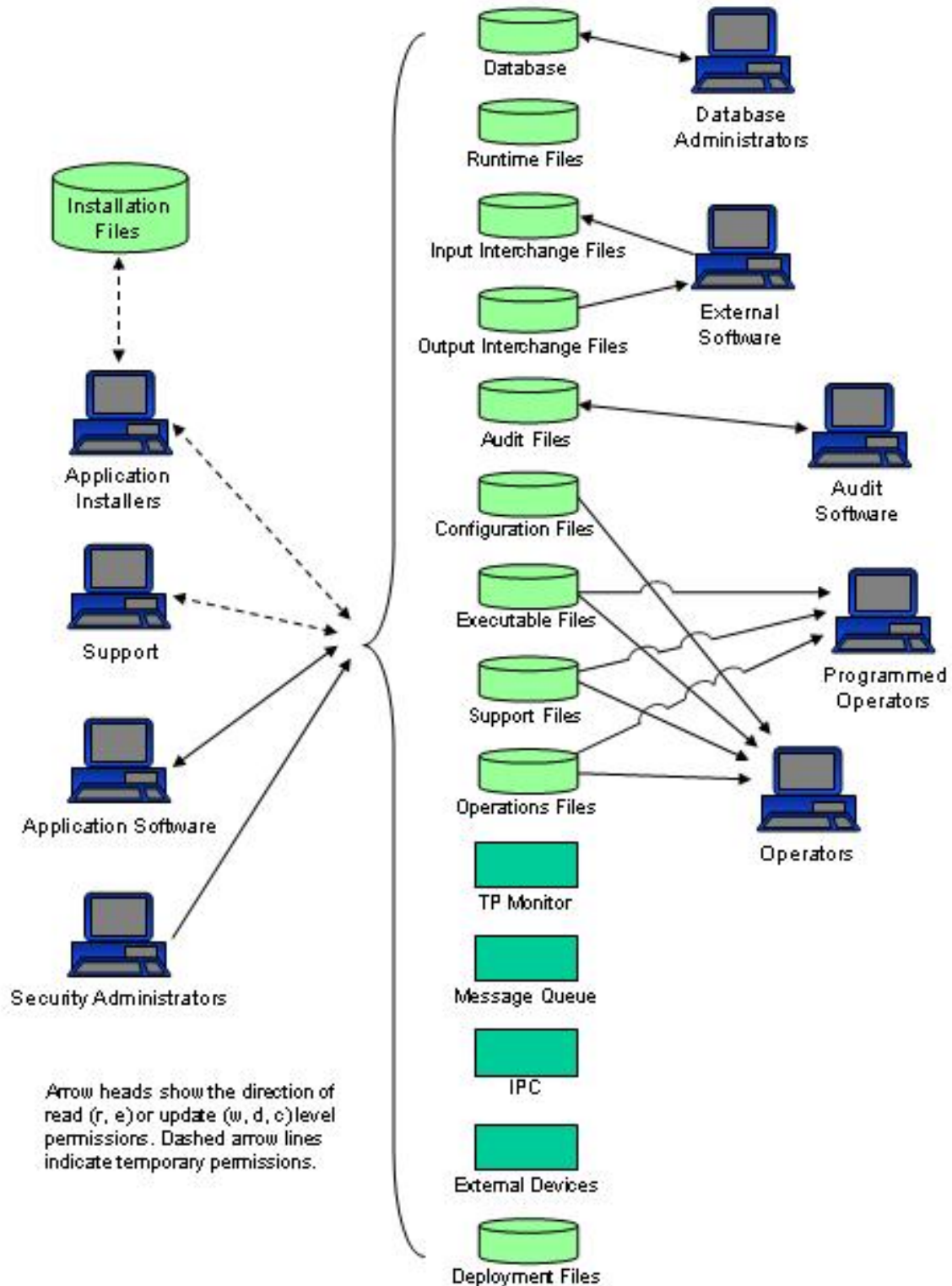


Figure 1 Actor permissions for system object types

Table 2 BASE24 System Object Locations

In the following table, locations specified in *italics* are symbolic names for configurable locations which will vary from customer to customer.

Generic System Objects	Equivalent BASE24 System Object Locations
Database	
Runtime Files	
Input Interchange Files	Authorization refresh files (full & partial)
Output Interchange Files	
Audit Files	Defined in TMF
Configuration Files	<i>\$<VOL>.AT60*</i> , <i>\$<VOL>.BA60*</i> , <i>\$<VOL>.PS60*</i> , <i>\$<VOL>.SW60*</i> , <i>\$<VOL>.BP60*</i> , <i>\$<VOL>.OK60*</i>
Executable Files	<i>\$<VOL>.XPNET</i> , <i>\$<VOL>.ACIUTILS</i> , <i>\$<VOL>.ACILIBI</i> , <i>\$<VOL>.AT60OBJ</i> , <i>*</i> , <i>\$<VOL>.BA60OBJ</i> , <i>\$<VOL>.PS60OBJ</i> , <i>\$<VOL>.SW60OBJ</i> , <i>\$<VOL>.BP60OBJ</i> , <i>\$<VOL>.OK60OBJ</i>
Support Files	<i>\$<VOL>.SCRIBE</i> ,
Operations Files	<i>\$System.system</i> , <i>\$<VOL>.ACIUTILS</i> , <i>\$<VOL>.ACILIBI</i>
TP Monitor	
Message Queue	
Shared Memory	
External Devices	ACI Communication Services for Advanced Networking (a.k.a. ICE-XS) resources
Installation Files	<i>\$System.system</i> , <i>\$<VOL>.SCRIBE</i>

2.7 Specific Rules for System Objects

The matrices in *Table 3 BASE24 System Object Security for BASE24 components* and *Table 4 BASE24 System Object Security for system users* show the standard set of permissions that should be applied to BASE24 system objects, at installation time. The rows indicate the various categories of system objects, and the columns indicate the job roles that may require access to system objects.

Key:

R – read allowed

W – write allowed

E – execute allowed

P – purge (delete) allowed

2.7.1 Temporary Access

Certain classes of user do not need permanent access, but may need access from time to time to perform their job function. Examples of these are:

1st, 2nd and 3rd level support

Application installers

1st, 2nd and 3rd level support access is described in *Table 4 BASE24 System Object Security for system users*, however it should be understood that this access does not need to be permanent. Depending on site policy, support's permissions can be granted on an individual and temporary basis, as the need arises. Since these are on production systems, access should be logged and then reviewed after the support incident is closed.

Likewise, BASE24 installers may be given very broad permissions during installations and upgrades, depending on the scale of the changes required. Once the changes are complete, the installers' permissions should be revoked.

2.7.2 Transfer of system objects off-site

A typical example of this would be when traces and dump/abend files are sent to ACI support for problem determination. ACI maintains a strict and formally documented internal Policy for handling of sensitive customer data such as this that has been entrusted to ACI. This situation is not addressed here, as we are only considering security at the customer's installation site.

2.7.3 Network Resources

The network resources used by BASE24 are many, varied and highly customizable, so it is not possible to document here all of the system objects that are involved. It is recommended that customers consult the documentation provided by platform and security software vendors to establish the best means for securing the communications controllers and system software through which networks are accessed. Security within the network itself is outside of the scope of this document.

2.7.4 Auditing and Audit Logging

Audit logging refers to the recording of activities that occur on the system. *Auditing* refers to the analysis of those records. This is in addition to the audit features of BASE24 itself. Audit logging is different from event logging, which refers to informatory, warning and error messages which are logged by application and system software itself during the normal course of processing to inform operators about the state of the software.

All user activity in the production environment, including activity by application programs, should be logged and audited, using the auditing facilities provided by the operating system and related tools. It therefore should always be possible to attribute any one action to an individual user or piece of application software.

It is necessary for the disk(s) on which the files APCF, ATT, IPCF, PDF, TCF and TRF reside be AUDITED and BASE24 requires that TMF be running.

2.7.5 System Object Owner

In most cases, the user id that is the owner of a file, directory or other system object is independent of the permissions specified in this manual and so has not been specified here. The owner of system objects should be set according to the security policy of the site where BASE24 is installed.

Table 3 BASE24 System Object Security for BASE24 components

Directories/Files/ Resources	Category	Application Software executables	UI
\$<VOL>.*OBJ	Executable & Configuration files	R, E	
\$<VOL>.*OBJ	Executable & Installation Files	E	
	Configuration Files	R	
	Installation Files		
	Support Files	W	W
	Installation Files		
	Configuration Files	R	
	Databases incl. Audit Files (tables)	R, W	W
	Support & Runtime Files	R, W	
	Output Files	R, W, D, C	
	Input Interchange Files		
Authorization File Refresh files (full and partial)	Input Interchange Files		
<i>dump locations</i>	Support Files	W	W
Shared memory	IPC	R, W, C	

Table 4 BASE24 System Object Security for system users

	Category	Operator	1st, 2nd, 3rd Support	Security Admin	Database Admin	Audit Software	Programmed Operators	Application Installers
\$<VOL>.*OBJ	Executable & Configuration files	R, E	R	C			E	R, W, E, D, C
\$<VOL>.*OBJ	Executable & Installation Files	E	R	C			E	R, W, E, D, C
	Configuration Files	R	R	C	R, W, D, C			R, W, D, C
	Audit Files		R	C		R		
	Installation Files		R	C				R, W, E, D, C
	Support Files	C	R	C			R	R, W, D, C
	Installation Files		R	C				R, W, E, D, C
	Configuration Files	R	R	C				R, W, D, C
	Databases incl. Audit Files (tables)		R	C	R, W, D, C	R		R, W, D, C
	Support & Runtime Files	C	R	C				R, W, D, C
	Output Files		R	C				R, D, C
Authorization File Refresh files (full and partial)	Input Interchange Files	R	R	C				R, W, D, C
dump locations	Support Files	C	R	C				
Shared memory	IPC							

ACI Worldwide, Inc.

2.7.6 Verification

Customers should verify their ACI product's system object security after installations and upgrades, after troubleshooting sessions, and periodically, to catch tampering. This can be done by using Table 3 BASE24 System Object Security for BASE24 components and Table 4 BASE24 System Object Security for system users above as a checklist to validate the system object security permissions for their installed BASE24 product.

In addition, the customer's own testing of BASE24 installations should now include system object security verification. For example, the customer's QA department could have a set of QA equivalents to production user id's, and with the appropriate production-equivalent access permissions set, use these to start, stop and run BASE24 and its components.

3 Command Quick Reference

The techniques for setting security, auditing and audit logging correctly on a modern production system require intimate knowledge of company security policy and platform hardware, software and network configuration and in depth knowledge of the vast security and audit services provided by modern, industrial-strength platforms.

This section gives only a summary of the platform-specific commands required to protect the system objects described in section 2.7 *Specific Rules for System Objects*. A simplified syntax is given for each command, with the minimum parameters required to set permissions, etc. as defined in this document. It is provided for the information of readers who do not have a security administration background on a platform, to give an indication of the steps involved in setting system object security. It is not a replacement for the advanced training, knowledge and experience that is required to effectively secure a commercial system.

3.1 HP NonStop

3.1.1 XPNET

3.1.1.1 XPNET Nodes

The XPNET node is the fundamental building block of an XPNET system. It consists of an XPNET process and its associated stations, lines, and application processes. The XPNET process and its application processes work together to accomplish message processing. XPNET nodes can be replicated and distributed as required to support processing and operational demands. Each XPNET node has the ability to communicate with other XPNET nodes. Because of these features, an XPNET system is highly scalable.

3.1.1.2 HP NonStop Systems

The *HP NonStop system* refers to the physical HP computer. The HP NonStop system executes the XPNET process and application software. HP NonStop systems communicate with one another using the HP NonStop Expand product, thereby functioning as a distributed processing system.

The HP NonStop network can consist of any number of HP NonStop systems and can include any available processor types or mix of processor types. The XPNET system can span multiple HP NonStop systems without affecting or altering the operational interfaces and without impacting connectivity between processes, stations, and lines in the configuration.

3.1.1.3 Network Control

Network control is a broad term encompassing all aspects of keeping an XPNET system operating efficiently according to the policies and operating procedures of an organization. This includes such functions as monitoring the general operation of the network; keeping the various lines up and stations running; starting and stopping processes, nodes, and links; and adding and reconfiguring destinations, devices, lines, links, nodes, processes, and stations. The basic tools of network control consist of operator interfaces, network control commands issued from these operator interfaces, event log files, and security features. These tools enable operators to effectively monitor and make online modifications to the XPNET system.

This section provides an introduction to network control functions and includes information on the following topics:

- The network control environment

- Operator interfaces
- Network security
- Event log files

Network control commands issued from operator or programmatic interfaces are processed by the NCP Server process, which is the control point for all network commands. After the NCP Server process validates a command for syntax, it sends the command to the NCPI Server process, which validates security and controls message flow between the NCP Server process and the XPNET process.

3.1.1.4 NCPI Server Process

The Network Control Point Interface (NCPI) Server process is a Pathway server that provides the programmatic interface to an XPNET node. It performs all security processing. There is one NCPI Server process per XPNET node. The NCPI Server process is responsible for validating operator command security.

This includes commands originating from all of the following sources:

- NCS screen
- NCPCOM utility
- Custom operator interfaces
- Custom programmatic interfaces
- HP NonStop or third-party developed programmatic interfaces (such as NonStop NET/MASTER or Programmatic Network Administrator)

The NCPI Server process accesses security files to determine if the operator issuing the command has a valid user identification and password and if the operator is granted access to the command issued and the XPNET node involved in the command.

3.1.1.5 Network Control Security

Network control command security is configurable. At its most basic level, access to network control commands is determined by access to a network control facility. If a user has access to a network control facility, then, without implementation of further security measures, he or she can execute any command. Two additional security measures can be taken, either separately or in conjunction with each other. These measures are to limit network control command access by command and to audit network control commands.

3.1.1.5.1 Command Access Limitations

Command access limitations allow you to specify for each user whether the user has access to each network control command. When you have enabled command access limitations, you assign a command profile to each user in his or her security record. The command profile indicates, for each network control command, whether the user has access to the network control command.

You enable command access limitation during configuration of the NCPI Server processes. In addition, two files are required to limit network control command access. These files are the Network Control Security Profile File (NCSP) and the Network Control Security Specification File (NCSS). Command access limitations are specified on the security screen that updates the NCSP. Options include the following:

- A = Audit access
- N = No access

U = Unrestricted
V = View unrestricted (with auditing)
Y = Access

An operator's security records are associated with a specific command profile on the security screen that updates the NCSS. These files are described below.

3.1.1.5.2 Command Auditing

Command auditing creates an audit trail of command activity. When you enable auditing by itself (without command access limitations), the XPNET process generates an event message each time someone executes a network control command. The event message identifies the user who issued the command, the XPNET node affected, the terminal the command was issued from, and the command that was performed.

You enable command auditing for nodes during configuration of the NCPI Server processes. This is used in conjunction with the command access limitations configured in the NCSP.

3.1.1.5.3 Using Command Access Limitations and Command Auditing Together

When you enable both command auditing and command access, the command profile controls not only whether the user can perform the command, but also whether execution of the command is audited. In this way, system administrators can audit selected commands or users. For example, new network control operators could be assigned a special command profile that gives audited access to some or all commands. Or, command profiles could be created that allow users to execute inquiry commands without being audited, while commands that change the system configuration are audited.

For more information on network control command security, see the *NET24-XPNET Network Control Operations Guide*

3.1.1.5.4 Network Control Security Profile File

The Network Control Security Profile File (NCSP) allows you to set up network control command profile records. The command profile records indicate whether each network control command can be executed and whether the command is audited when it is executed. After you have created a command profile and stored it in the NCSP, you can associate it with the security record for one or more users.

3.1.1.5.5 Network Control Security Specification File

The security screen that accesses the Network Control Security Specification File (NCSS) allows you to specify the network control command profile to be associated with a user's security record. The NCSS allows a single command profile to be assigned to multiple users. The command profile must be added to the NCSP before the profile can be assigned to a user.

The user's security record contains the following:

- User alias
- Node name
- Command profile name
- Password
- Network control access limitations by time of day, day of the week, and last date allowed
- Password requirements for minimum and maximum length, number of alpha and numeric characters, frequency of password changes, password reuse limitations, and maximum invalid password attempts allowed

3.1.2 Guardian

NSK supports two main methods for securing system objects:

Guardian security (for disk file and file patterns only) using the FUP utility

Safeguard (for files, volumes and subvolumes, devices and subdevices, processes and subprocesses, OBJECTTYPE)

As Safeguard is an extension to Guardian security, it is recommended and this section will assume that it is being used. Unless stated otherwise all commands given in the subsections below are Safeguard commands.

Safeguard commands are entered using the Guardian SAFECOM utility. To use SAFECOM, a user must have EXECUTE (E) authority for SAFECOM.

Guardian **user names** are of the form *groupname.membername*. User names all have an equivalent **user id** in the form *groupnumber,membernumber*. The *groupnumber* is associated with the *groupname* and is in the range 0 to 255. It is known as the **administrative group** of the user. The *membernumber* is associated with the *membername* and is in the range 0 to 255. A *membernumber* of 255 is always the manager of the group. A *groupnumber* of 255 is always the SUPER group. Users can be members of other groups, apart from their administrative group.

A Safeguard **authority** maps to the system object permissions defined in this document, as follows:

Table 3-1 Safeguard authorities

System Object Permission	Equivalent Safeguard authority	Safeguard description	Applicable system object types
R	R or READ	Open a system object for input operations.	All
W	W or WRITE	Open a system object for output operations.	All
	C or CREATE	Create a disk file with the specified name. Create a disk file on the (sub)volume. Create a process with the specified name.	Disk file Files within a volume Files within a subvolume Process
E	E or EXECUTE	Run a disk file.	Disk file Files within a volume Files within a subvolume
D	P or PURGE	Stop a process. Delete a disk file.	Disk file Files within a volume Files within a subvolume Process
C	O or OWNER	Manage the system	All

		object's authorization records in Safeguard.	
--	--	--	--

3.1.2.1 Defining groups (roles)

ADD GROUP *groupname*, NUMBER *groupnumber*, OWNER *ownerid*

See [3], chapter 3.

At the time of writing, only “administrative” groups can be used in Safeguard access control lists. Non-administrative groups can only be used for file sharing. To be able to serve as an administrative group, the following must be true:

groupname must be from one to eight alphanumeric characters, and the first character must be alphabetic

groupnumber must be in the range 0 to 255

A group is activated as an administrative group if at least one user has been added with the same *groupname* as follows:

```
ADD USER groupname.membername, groupnumber,membernumber, OWNER
ownerid, PASSWORD password, GUARDIAN SUBVOLUME $disk.subvol,
DEFAULT-PROTECTION ACCESS \group-num,* *
```

To add a user that activates the group as an administrative group, *groupname* and *groupnumber* in the ADD USER command must be the same as that specified in the ADD GROUP command.

See [3], chapter 2.

3.1.2.2 Connecting a user to a group

Users always belong to the administrative group that forms the first part of their user name and user id. A user can be added to other groups with the following command:

```
ALTER GROUP groupname, MEMBER usergroupname.usermembername
```

See [4], chapter 7.

3.1.2.3 Securing a subvolume

A specific group can be allowed to access a subvolume, by using the ADD SUBVOLUME command:

```
ADD SUBVOLUME $disk.subvol, OWNER groupname.membername, ACCESS GROUP
groupname authoritylist
```

Where *authoritylist* is either a single *authority*

or a list in the form *(authority, authority, ...)*

Refer **Table 3-1** for the possible values for *authority*.

See [4], chapter 9.

3.1.2.4 Securing a disk file

3.1.2.4.1 Files that already exist

A specific group can be allowed to access one or more existing disk files, by using the ADD DISKFILE command:

```
ADD DISKFILE $disk.subvol.file, OWNER groupname.membername, ACCESS  
GROUP groupname.authoritylist, PROGID {ON|OFF}, CLEARONPURGE  
{ON|OFF}
```

Where *authoritylist* is either a single *authority*
or a list in the form *(authority, authority, ...)*

Refer **Table 3-1** for the possible values for *authority*.

PROGID is relevant only for executable files. When set to **ON**, any process executed from the file runs under the user id specified as **OWNER**. When set to **OFF** or if this parameter is omitted, any process executed from the file runs under the user id of the user that starts it.

CLEARONPURGE, if specified and set to **ON**, performs a secure delete for the file. That is, if/when the file is deleted, the data pages of file will be physically cleared on disk.

Note that *\$disk.subvol.file* can contain wildcard characters.

3.1.2.4.2 Files that do not yet exist

A specific group can be allowed to access one or more disk files that do not yet exist, by using the **ADD DISKFILE-PATTERN** command:

```
ADD DISKFILE-PATTERN $disk.subvolpattern.filepattern, OWNER  
groupname.membername, ACCESS GROUP groupname.authoritylist
```

Parameters are as described in section 3.1.2.4.1 above. Note that the **CLEARONPURGE** and **PROGID** attributes are not available.

See [4], chapter 8.

3.1.2.5 Securing a process or subprocess

A specific group can be allowed to access a process or subprocess, by using the **ADD PROCESS** or **ADD SUBPROCESS** command:

```
ADD PROCESS $processname, OWNER groupname.membername, ACCESS GROUP  
groupname.authoritylist
```

Where *authoritylist* is either a single *authority*
or a list in the for *(authority, authority, ...)*

Refer **Table 3-1** for the possible values for *authority*. **PURGE** authority allows a user to stop a process. **EXECUTE** authority is not allowed for processes or subprocesses.

\$processname cannot contain wildcard characters.

The **ADD SUBPROCESS** command takes the same parameters as the **ADD PROCESS** command, i.e.

```
ADD SUBPROCESS $processname.#qual, OWNER groupname.membername,  
ACCESS GROUP groupname.authoritylist
```

\$processname.#qual cannot contain wildcard characters.

See [4], chapter 11.

3.1.2.6 Securing a device or subdevice

A specific group can be allowed to access an NSK device or subdevice (terminals, tape drives, communication lines, printers, etc.), by using the **ADD DEVICE** or **ADD SUBDEVICE** command:

```
ADD DEVICE $devicename, OWNER groupname.membername, ACCESS GROUP  
groupname.authoritylist
```

Where *authoritylist* is either a single *authority*

or a list in the form

(*authority, authority, ...*)

authority may be **only** READ, WRITE or OWNER. PURGE and EXECUTE are not allowed for devices and subdevices. Refer **Table 3-1** for more information about these values.

\$devicename cannot contain wildcard characters.

The ADD SUBDEVICE command takes the same parameters as the ADD DEVICE command, i.e.

ADD SUBDEVICE *\$devicename.#qual*, OWNER *groupname.membername*, ACCESS GROUP *groupname authoritylist*

\$devicename.#qual cannot contain wildcard characters.

See [4], chapter 11.

3.1.2.7 Audit logging against Guardian system objects

Audit logging of all access attempts to system objects is strongly recommended.

Under Safeguard, audit logging can be turned on for all access attempts to system objects by adding the following parameters to the Safeguard commands for securing system objects (i.e. ADD DISKFILE, ADD DISKFILE-PATTERN, ADD PROCESS, ADD SUBPROCESS, ADD VOLUME, ADD SUBVOLUME, ADD DEVICE and ADD SUBDEVICE).

AUDIT-ACCESS-PASS ALL, AUDIT-ACCESS-FAIL ALL, AUDIT-MANAGE-PASS ALL, AUDIT-MANAGE-FAIL ALL

See [4], chapter 1 (pp. 1-7 to 1-8) and the specific command documentation referred to in the sections above.

4 Reference and Bibliography

- [1] shm_open(), *The Open Group Base Specifications Issue 6*, available at http://www.opengroup.org/onlinepubs/000095399/functions/shm_open.html
- [2] semget(), *The Open Group Base Specifications Issue 6*, available at <http://www.opengroup.org/onlinepubs/000095399/functions/semget.html>
- [3] *Safeguard Administrator's Manual*, Hewlett-Packard 523317-014, August 2007, Safeguard G07, H04, available at <http://docs.hp.com/en/523317-014/523317-014.pdf>
- [4] *Safeguard Reference Manual*, Hewlett-Packard 520618-014, August 2007, Safeguard G07, H04, available at <http://docs.hp.com/en/520618-014/520618-014.pdf>
- [5] *Safeguard User's Guide*, Hewlett-Packard 422089-010, August 2007, Safeguard G07, H06, available at <http://docs.hp.com/en/422089-010/422089-010.pdf>
- [6] S. Shepler, B. Callaghan, D. Robinson, R. Thurlow, C. Beame, M. Eisler, D. Noveck, *Network File System (NFS) version 4 Protocol*, RFC 3530, April 2003, available at: <http://www.rfc-editor.org/rfc/pdfrfc/rfc3530.txt.pdf>
- [7] *Introduction Manual, NET24-XPNET*, ACI publication number XN-NT000-04, Jun-2006